

2. Estruturas Condicional, Loops, Listas, Operações Lógicas e bit a bit;

▼ Estruturas de Controle (condicional)

Condicionais

Em programação, nem sempre o código deve seguir um único caminho.

Muitas vezes, é necessário **tomar decisões** com base em dados informados pelo usuário ou resultados de cálculos.

As **estruturas de controle condicionais** permitem que o programa execute ações diferentes conforme uma condição seja verdadeira ou falsa.

O que são Estruturas Condicionais

Estruturas condicionais avaliam **expressões lógicas** e, a partir do resultado, determinam qual bloco de código será executado.

Essas estruturas tornam os programas:

- Mais dinâmicos
 - Interativos
 - Adaptáveis a diferentes situações
-

Operadores Relacionais e Lógicos

Antes de usar condicionais, é importante compreender os operadores mais comuns.

Operadores Relacionais

Operador	Significado
<code>==</code>	Igual
<code>!=</code>	Diferente

Operador	Significado
>	Maior
<	Menor
>=	Maior ou igual
<=	Menor ou igual

Operadores Lógicos

Operador	Significado
and	E
or	Ou
not	Negação

Esses operadores são usados dentro das condições do `if`, `elif`, `while` e outras estruturas.

Estrutura `if-else`

A estrutura `if-else` é utilizada quando há **duas possibilidades**: uma condição verdadeira ou falsa.

Exemplo: Dia da Semana

```
dia = input("Digite o dia da semana: ").lower()

if dia == "segunda":
    print("Início da semana, foco total!")
else:
    print("Dia comum.")
```

Neste exemplo:

- O programa verifica se o valor de `dia` é `"segunda"`.
- Se for verdadeiro, executa o bloco do `if`.
- Caso contrário, executa o bloco do `else`.

Exemplo Clássico: Maioridade

Situação lógica:

Se a idade for maior ou igual a 18, então a pessoa é maior de idade.

Caso contrário, é menor de idade.

Implementação em Python:

```
idade = int(input("Digite sua idade: "))

if idade >= 18:
    print("Você é maior de idade.")
else:
    print("Você é menor de idade.")
```

Esse tipo de estrutura é muito comum em validações e regras de negócio.

Estrutura **if-elif-else**

O **if-elif-else** é usado quando existem **mais de duas possibilidades** e apenas uma delas deve ser executada.

O Python avalia as condições **de cima para baixo** e executa o primeiro bloco verdadeiro encontrado.

Exemplo: Dia da Semana

```
dia = input("Digite o dia da semana: ").lower()

if dia == "segunda":
    print("Início da semana, foco total!")
elif dia == "sexta":
    print("Sextou!")
elif dia == "sábado" or dia == "domingo":
    print("É fim de semana, aproveite!")
else:
    print("Dia comum.")
```

Observações importantes:

- Apenas um bloco será executado.

- O `else` é opcional, mas recomendado para tratar casos inesperados.

Estrutura `match-case` (Python 3.10 ou superior)

O `match-case` é uma alternativa mais moderna e organizada para cadeias longas de `if-elif-else`.

Ele funciona de forma semelhante a um "comparador de casos".

Exemplo: Dia da Semana

```
dia = input("Digite o dia da semana: ").lower()

match dia:
    case "segunda":
        print("Início da semana, foco total!")
    case "sexta":
        print("Sextou!")
    case "sábado" | "domingo":
        print("É fim de semana, aproveite!")
    case _:
        print("Dia comum.")
```

Pontos importantes:

- O valor é comparado diretamente com cada `case`.
- O símbolo `|` representa alternativa lógica.
- O `_` funciona como um curinga, equivalente ao `else`.

Comparação entre as Estruturas

Estrutura	Quando usar
<code>if-else</code>	Duas possibilidades
<code>if-elif-else</code>	Múltiplas condições encadeadas
<code>match-case</code>	Muitos casos fixos e bem definidos

Nem sempre o `match-case` é necessário. Para casos simples, `if-else` continua sendo a melhor opção.

Exemplos Práticos Adicionais

Verificar se um número é positivo, negativo ou zero

```
num = int(input("Digite um número: "))

if num > 0:
    print("Número positivo")
elif num < 0:
    print("Número negativo")
else:
    print("Número igual a zero")
```

Verificar aprovação de aluno

```
nota = float(input("Digite a nota: "))

if nota >= 7:
    print("Aluno aprovado")
elif nota >= 5:
    print("Aluno em recuperação")
else:
    print("Aluno reprovado")
```

Boas Práticas

- Sempre indentar corretamente os blocos de código.
- Evitar condições muito longas ou confusas.
- Preferir clareza à complexidade.
- Usar `match-case` apenas quando fizer sentido.

Resumo Rápido

- Estruturas condicionais permitem decisões no código.
- `if-else` trata duas possibilidades.

- `if-elif-else` trata múltiplos cenários.
 - `match-case` organiza comparações diretas.
 - Operadores relacionais e lógicos são fundamentais.
-

Fontes e Leituras Recomendadas

- Documentação oficial do Python. Controle de fluxo
<https://docs.python.org/pt-br/3/tutorial/controlflow.html>
 - Documentação oficial. `match-case`
https://docs.python.org/pt-br/3/reference/compound_stmts.html#the-match-statement
 - Real Python. Conditional Statements
<https://realpython.com/python-conditional-statements/>
 - W3Schools. Python If Else
https://www.w3schools.com/python/python_conditions.asp
-

Observação crítica, como colega

Muitos materiais apresentam condicionais sem explicar **ordem de avaliação** e **exclusividade dos blocos**. Isso gera bugs silenciosos. Aqui o foco é clareza lógica, não quantidade de exemplos.

Se quiser, posso:

- Padronizar essa apostila com a de **laços** em um único PDF
- Criar uma **versão resumida de consulta rápida**
- Montar **exercícios sem resolução**
- Adaptar para **informática básica, técnico ou ensino médio**

Diga como você pretende usar o material que eu ajusto o nível fino.

Na programação, muitas vezes precisamos que o programa tome decisões com base em condições. Para isso, usamos **estruturas de controle**, que permitem diferentes ações dependendo de uma situação.



ex.: Vamos utilizar as abordagens: `if-else`, `if-elif-else` e `match-case` para verificar o dia da semana:

▼ `if-else`

```
dia = input("Digite o dia da semana: ").lower()

if dia == "segunda":
    print("Início da semana, foco total!")
else:
    print("Dia comum.")
```

Neste caso, o código verifica se o valor de `dia` é igual a "segunda". Se for verdadeiro, imprime a mensagem, caso contrário, imprime "Dia comum".

▼ Por exemplo, ao verificar se uma pessoa é maior de idade, podemos usar a estrutura condicional:



"Se a idade for maior ou igual a 18, então ela é maior de idade. Caso contrário, é menor de idade."

Em Python, isso seria assim:

```
idade = int(input("Digite sua idade: "))
if idade >= 18:
    print("Você é maior de idade.")
else:
    print("Você é menor de idade.")
```

Essas estruturas tornam os programas mais dinâmicos e interativos.

▼ `if-elif-else`

```
dia = input("Digite o dia da semana: ").lower()

if dia == "segunda":
```

```
print("Início da semana, foco total!")
elif dia == "sexta":
    print("Sextou!")
elif dia == "sábado" or dia == "domingo":
    print("É fim de semana, aproveite!")
else:
    print("Dia comum.")
```

O `if-elif-else` é usado quando há múltiplas condições a serem verificadas. O código verifica cada caso, e assim que encontra um que seja verdadeiro, ele executa o bloco correspondente e pula os outros.

▼ `match-case` (Python 3.10 +)

```
dia = input("Digite o dia da semana: ").lower()

match dia:
    case "segunda":
        print("Início da semana, foco total!")
    case "sexta":
        print("Sextou!")
    case "sábado" | "domingo":
        print("É fim de semana, aproveite!")
    case _:
        print("Dia comum.")
```

No `match-case`, o valor de `dia` é comparado diretamente com os casos definidos. O `_` funciona como um curinga, e se nenhum caso anterior for verdadeiro, o código executa esse bloco, similar ao `else`.

▼ Estruturas de Repetição | Laços de Repetição | Loops

Em programação, muitas tarefas precisam ser executadas várias vezes.

Para evitar repetir código manualmente, utilizamos **estruturas de repetição**, também chamadas de **laços** ou **loops**.

Em Python, os laços permitem executar um mesmo bloco de código de forma automática, controlada e eficiente.

O que são Laços de Repetição

Um laço de repetição é uma estrutura que executa um conjunto de instruções **mais de uma vez**, de acordo com uma regra definida.

Eles são usados para:

- Contagens
- Percorrer sequências de dados
- Repetir ações até uma condição ser satisfeita
- Criar jogos, menus e validações

Tipos de Laços em Python

Python possui dois laços principais:

Laço `for`

O `for` é utilizado quando existe uma **sequência definida** de valores ou quando sabemos quantas vezes o código deve se repetir.

Ele percorre automaticamente cada elemento de uma sequência.

```
for i in range(3):  
    print(i)
```

Laço `while`

O `while` executa um bloco de código **enquanto uma condição for verdadeira**.

A repetição continua até que a condição se torne falsa.

```
contador = 1  
while contador <= 5:  
    print(contador)  
    contador += 1
```

A Função `range()`

A função `range()` gera uma sequência de números e é muito usada com o laço `for`.

Sintaxe

```
range(inicio, fim, passo)
```

- `inicio` (opcional): valor inicial. Padrão é 0.
- `fim`: valor final, não incluído.
- `passo` (opcional): incremento entre os valores.

Exemplos

```
range(5)      # 0, 1, 2, 3, 4
range(1, 6)   # 1, 2, 3, 4, 5
range(0, 10, 2) # 0, 2, 4, 6, 8
```

Exemplos de Uso

Exemplo com `for`

```
for i in range(5):
    print("Passo", i)
```

Exemplo com `while`

```
i = 0
while i < 5:
    print("Passo", i)
    i += 1
```

Palavras-chave Importantes

`break`

Interrompe o laço imediatamente.

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

continue

Pula a iteração atual e segue para a próxima.

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

Exercícios Resolvidos

Contar de 1 a 10 usando **for**

```
for i in range(1, 11):
    print(i)
```

Contar de 10 a 1 usando **while**

```
i = 10
while i >= 1:
    print(i)
    i -= 1
```

Imprimir apenas números pares de 0 a 20

```
for i in range(0, 21, 2):
    print(i)
```

Tabuada de um número informado pelo usuário

```
num = int(input("Digite um número: "))
for i in range(1, 11):
    print(f"{num} x {i} = {num * i}")
```

Desafios Propostos

Cofre Secreto

O programa solicita a senha até que a senha correta seja digitada.

```
senha = "python123"
tentativa = ""

while tentativa != senha:
    tentativa = input("Digite a senha: ")
    if tentativa != senha:
        print("Acesso negado!")

print("Acesso liberado!")
```

Número Mágico

O usuário deve adivinhar um número gerado aleatoriamente.

```
import random

numero = random.randint(1, 10)
chute = 0

while chute != numero:
    chute = int(input("Adivinhe o número entre 1 e 10: "))
    if chute < numero:
        print("Muito baixo!")
    elif chute > numero:
        print("Muito alto!")
```

```
print("Parabéns! Você acertou!")
```

Resumo Rápido

- Use `for` quando houver uma sequência definida.
- Use `while` quando a repetição depender de uma condição.
- `range()` gera sequências numéricas.
- `break` encerra o laço.
- `continue` pula para a próxima repetição.

Fontes e Leituras Recomendadas

- Documentação oficial do Python. Estruturas de controle
<https://docs.python.org/pt-br/3/tutorial/controlflow.html>
- Documentação da função `range()`
<https://docs.python.org/pt-br/3/library/stdtypes.html#range>
- Real Python. For Loops em Python
<https://realpython.com/python-for-loop/>
- W3Schools. While Loops
https://www.w3schools.com/python/python_while_loops.asp

▼ Operações Bit a Bit (Bitwise) em Python

Material de Apoio

As operações **bit a bit** trabalham diretamente com os **bits** que compõem os números inteiros.

Em vez de operar sobre valores decimais completos, essas operações atuam sobre cada bit individualmente.

Essas operações são muito utilizadas em:

- Programação de baixo nível
- Otimização de desempenho

- Manipulação de permissões e flags
- Sistemas embarcados
- Redes e criptografia
- Máscaras de bits

O que são Bits

Um **bit** é a menor unidade de informação em computação e pode assumir apenas dois valores:

- 0
- 1

Números inteiros são armazenados internamente em **binário** (base 2).

Exemplo:

Decimal	Binário
5	0101
6	0110
10	1010

Em Python, podemos visualizar o valor binário usando a função `bin()`:

```
bin(10)
# '0b1010'
```

Operadores Bit a Bit em Python

Python possui operadores específicos para manipular bits.

Operador	Nome	Descrição
&	AND	E bit a bit
	OR	OR
^	XOR	OU exclusivo
~	NOT	Negação
<<	Shift Left	Deslocamento à esquerda

Operador	Nome	Descrição
>>	Shift Right	Deslocamento à direita

AND (&) – E Bit a Bit

O operador & retorna 1 apenas quando **ambos os bits são 1**.

Exemplo:

```
a = 5  # 0101
b = 3  # 0011

resultado = a & b
print(resultado)
```

Análise bit a bit:

```
  0101
& 0011
-----
  0001
```

Resultado em decimal: 1

OR (|) – OU Bit a Bit

O operador | retorna 1 quando **peelo menos um dos bits é 1**.

```
a = 5  # 0101
b = 3  # 0011

resultado = a | b
print(resultado)
```

```
  0101
| 0011
```

```
-----  
0111
```

Resultado em decimal: 7

XOR (^) – OU Exclusivo

O operador ^ retorna 1 quando **os bits são diferentes**.

```
a = 5  # 0101  
b = 3  # 0011  
  
resultado = a ^ b  
print(resultado)
```

```
0101  
^ 0011  
-----  
0110
```

Resultado em decimal: 6

Uso comum: alternar estados, criptografia simples, flags.

NOT (~) – Negação Bit a Bit

O operador ~ inverte todos os bits do número.

```
a = 5  
print(~a)
```

Em Python, o resultado é negativo devido ao uso de **complemento de dois**.

```
~5 = -6
```

Regra prática:

```
~x == -(x + 1)
```


Esse operador exige atenção, pois não se limita a um número fixo de bits.

Shift Left (<<) – Deslocamento à Esquerda

Desloca os bits para a esquerda, inserindo zeros à direita.

Cada deslocamento equivale a **multiplicar por 2**.

```
a = 5 # 0101  
print(a << 1)
```

0101 << 1 = 1010

Resultado: 10

```
a << 2 # multiplica por 4
```

Shift Right (>>) – Deslocamento à Direita

Desloca os bits para a direita, descartando bits à direita.

Cada deslocamento equivale a **dividir por 2** (parte inteira).

```
a = 10 # 1010  
print(a >> 1)
```

1010 >> 1 = 0101

Resultado: 5

Comparação: Operadores Lógicos vs Bit a Bit

Tipo	Operador	Exemplo
Lógico	and , or , not	True and False
Bit a Bit	& , ^	1 & 1, ~1

⚠ Erro comum: confundir and com &.

```
True and False # False
5 & 3          # 1
```

Aplicações Práticas

Verificar se um número é par

```
num = 10

if num & 1 == 0:
    print("Par")
else:
    print("Ímpar")
```

Trabalhar com Flags (Permissões)

```
LEITURA = 1    # 0001
ESCRITA = 2     # 0010
EXECUCAO = 4   # 0100

permissao = LEITURA | ESCRITA

if permissao & LEITURA:
    print("Permissão de leitura")
```

Esse padrão é amplamente usado em sistemas e APIs.

Boas Práticas

- Use operações bit a bit apenas quando fizer sentido
- Prefira clareza à otimização prematura
- Documente bem o código ao usar máscaras de bits
- Evite usar `~` sem entender complemento de dois

Resumo Rápido

- Operações bit a bit trabalham diretamente com bits
 - `&` verifica bits iguais a 1
 - `|` verifica pelo menos um bit igual a 1
 - `^` verifica bits diferentes
 - `~` inverte todos os bits
 - `<<` multiplica por potências de 2
 - `>>` divide por potências de 2
-

Fontes e Leituras Recomendadas

- Artigo. Operações bit a bit em Python
<https://ealexbarros.medium.com/operacoes-bit-a-bit-em-python-e75624ce0611>
 - Documentação oficial do Python. Operadores
<https://docs.python.org/pt-br/3/reference/expressions.html#binary-bitwise-operations>
 - Real Python. Bitwise Operators
<https://realpython.com/python-bitwise-operators/>
 - GeeksForGeeks. Bitwise Operators in Python
<https://www.geeksforgeeks.org/python-bitwise-operators/>
-

Observação crítica, como colega

Bitwise é frequentemente ensinado cedo demais ou sem contexto. O resultado é confusão. Esse conteúdo faz sentido **após operadores, condicionais e tipos inteiros**, não antes.

Se quiser, posso:

- Criar uma **tabela visual binário vs decimal**
- Montar **exercícios graduais**
- Integrar com **condicionais e flags**

- Adaptar para **informática básica ou técnico**

Você decide o próximo refinamento.